第十五讲教学设计：用户认证与权限控制 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: 用户认证与权限控制（会话安全与基于角色的访问控制）
- **授课时长**: 20分钟
- **适用对象**: 已掌握Session、Filter、DAO与事务基础，准备完善安全治理的学生

一、教学目标

知识目标

1. 区分认证（Authentication）与授权（Authorization）的概念与流程。
2. 掌握基于会话（Session-Based）的登录态维护与注销（Logout）机制。
3. 掌握基于角色（RBAC: Role-Based Access Control）的访问控制思路与过滤器（Filter）实现方式。
4. 了解密码安全（Password Security）：摘要与散列（Hash: 推荐BCrypt/Argon2；示例用SHA-256演示原理）。

能力目标

1. 能实现登录、登出、会话保存登录用户信息与基本的失败提示。
2. 能编写“基于角色”的权限过滤器，对 `/admin/*` 等受限路径实施访问控制。
3. 能在JSP视图层按角色动态渲染菜单/按钮。

素质目标

1. 树立“最小权限原则”与“默认拒绝”的安全意识。
2. 形成合规观与责任心：妥善保管用户敏感信息，不在日志中明文打印密码等。
3. **[思政融合]** 以“制度先行、权责明晰”比喻RBAC，强调依法治网、守护秩序与公平正义。

二、教学重点与难点

教学重点

- 登录流程（认证）与基于角色的访问控制（授权）串联实现。
- 权限过滤器与白名单放行策略，403拒绝与友好错误页。
- 密码散列校验的基本原理与实现要点。

教学难点

- 会话中用户对象与角色信息的管理、更新与清理。
- 授权规则的可配置化与可扩展性（路径→角色映射）。
- 安全细节：重复提交、固定会话攻击、越权风险识别。

三、教学内容与时间安排

第一部分：问题引入——“谁可以进后台？” (4分钟)

内容要点

1. 展示未授权用户直接访问 `/admin/panel` 的风险与后果。
2. 提出目标：仅认证通过且具备ADMIN角色的用户可访问。
3. 串联已有技术：Session + Filter + DAO + Service。

教学方法

- 场景驱动 + 目标拆解。

第二部分：新知讲解——登录认证与RBAC实现 (10分钟)

内容要点

1. 密码散列工具（演示用SHA-256，生产建议BCrypt）：

```java
import java.security.MessageDigest;
public class PasswordUtil {
    public static String sha256(String s) {
        try {
            MessageDigest md = MessageDigest.getInstance("SHA-256");
            byte[] out = md.digest(s.getBytes("UTF-8"));
            StringBuilder sb = new StringBuilder();
            for (byte b : out) sb.append(String.format("%02x", b));
            return sb.toString();
        } catch (Exception e) { throw new RuntimeException(e); }
    }
    public static boolean matches(String raw, String hash) { return
sha256(raw).equals(hash); }
}
```

2. 登录控制器：

```java
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    private final UserService userService = new UserService();
    @Override protected void doPost(HttpServletRequest req, HttpServletResponse
resp)
            throws IOException, ServletException {
        String username = req.getParameter("username");
        String pwd = req.getParameter("password");
        User u = userService.login(username, pwd);
        if (u != null) {
            req.getSession().setAttribute("user", u);
            resp.sendRedirect(req.getContextPath() + "/index.jsp");
        } else {
            req.setAttribute("error", "用户名或密码错误");
            req.getRequestDispatcher("/jsp/login.jsp").forward(req, resp);
        }
```

```
    }
}
```

3. 注销控制器：

```
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
    @Override protected void doGet(HttpServletRequest req, HttpServletResponse
resp) throws IOException {
        HttpSession session = req.getSession(false);
        if (session != null) session.invalidate();
        resp.sendRedirect(req.getContextPath() + "/jsp/login.jsp");
    }
}
```

4. 基于角色的权限过滤器：

```
@WebFilter(urlPatterns = {"/admin/*"})
public class RoleBasedAuthFilter implements Filter {
    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain
chain)
            throws IOException, ServletException {
        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;
        HttpSession session = request.getSession(false);
        User u = (session == null) ? null : (User) session.getAttribute("user");
        if (u != null && "ADMIN".equals(u.getRole())) {
            chain.doFilter(req, res);
        } else {
            response.sendError(HttpServletResponse.SC_FORBIDDEN); // 403
        }
    }
}
```

5. 视图层按角色渲染（JSTL）：

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<c:if test="${sessionScope.user != null}">
    欢迎：${sessionScope.user.username}
    <a href="${pageContext.request.contextPath}/logout">退出</a>
</c:if>
<c:if test="${sessionScope.user.role == 'ADMIN'}">
    <a href="${pageContext.request.contextPath}/admin/panel">管理后台</a>
</c:if>
```

6. 403友好页（可选）：

```
<!-- web.xml -->
<error-page>
    <error-code>403</error-code>
    <location>/jsp/403.jsp</location>
</error-page>
```

教学方法

- 代码走查 + 联调演示（未登录/普通用户/管理员三种路径）。

第三部分：实战——给后台上锁 (4分钟)

内容要点

1. 准备两个用户：`admin/ADMIN` 与 `tom/USER`（密码存储为散列）。
2. 验证三条路径：
    - 未登录访问 `/admin/panel` → 403或重定向登录。
    - 普通用户登录后访问 `/admin/panel` → 403。
    - ADMIN用户登录访问 → 正常打开。
3. 在JSP菜单中按角色展示“管理后台”入口。

教学方法

- 现场联调 + 角色切换验证。

第四部分：课堂总结与展望 (2分钟)

总结要点

1. 认证负责“你是谁”，授权负责“你能做什么”。
2. RBAC通过角色简化授权管理；过滤器是落地RBAC的有效手段。
3. 密码应使用强算法与随机盐存储，避免明文与弱摘要。

展望（承上启下）

- 下一讲将把“分层 + 事务 + 权限”的能力融会贯通，用于更完整的业务场景与性能优化。

四、教学方法与手段

教学方法

1. **场景驱动 (Scenario-Driven)**: 从"谁可以进后台"安全需求入手
2. **代码走查 (Code Walkthrough)**: 讲解认证/授权实现
3. **联调演示 (Live Testing)**: 三种角色路径验证
4. **安全案例 (Security Case)**: SQL注入与密码散列
5. **双语教学法 (Bilingual Teaching)**:
    - 课堂采用英文授课，关键技术术语提供中英文对照
    - 重要概念（Authentication, Authorization, RBAC, Session-Based, Password Hash）首次出现时给出英文表达和中文释义
    - 鼓励学生使用英文术语描述安全机制

    - 代码注释和变量命名使用规范英文

教学手段

- IDE (IntelliJ IDEA/Eclipse)
- Tomcat服务器 (Tomcat Server)
- 浏览器开发者工具 (Browser DevTools)
- PPT/Marp课件 (Presentation Slides)

五、教学评价

- **过程性评价**: 能否清晰描述认证与授权的区别；能否解释过滤器拒绝的逻辑与白名单策略。
- **终结性评价**: 能否实现登录/登出与 `/admin/*` 的基于角色访问控制，并在JSP中按角色渲染菜单。

六、课后拓展

思考题

1. 如何防御“固定会话攻击”（Session Fixation）？登录成功后是否应更换 `JSESSIONID`？
2. 如果一个页面既允许ADMIN也允许MANAGER访问，你会如何编排过滤器的判定逻辑与配置？

实践作业

1. 将密码散列替换为BCrypt（如 `BCrypt.checkpw`），对比彩虹表攻击下的安全性差异。
2. 为 `/report/*` 新增角色 `MANAGER` 的访问控制，并在JSP中新增相应的菜单展示。